MODULE 17 · BLOCK 4 - FUNCTIONS AND EXCEPTIONS

Functions II: passing arguments

Parameters and arguments, positional and keyword passing, default values, and what a function can change in the caller's data.

BLOCK 4 of 4 EXAM WEIGHT 28% MODULE 17 of 28

admin@sthithapragna.com

Passing an argument works exactly like **parameter = argument**. The function gets another name for the same object, so it can change a list, but never the caller's variable.

Parameters are in the def. Arguments are in the call

```
1  def subtract(a, b):          # a and b are parameters
2      return a - b
3
4  print(subtract(5, 2))        # 5 and 2 are arguments: 3
```

1 Parameters are names in the header. They exist only while the function runs.

4 Arguments are the values supplied in the call.

By position, by name, or both

```
1  def subtract(a, b):
2      return a - b
3  print(subtract(5, 2))          # positional: a=5, b=2 gives 3
4  print(subtract(b=5, a=2))      # keyword: a=2, b=5 gives -3
5  print(subtract(5, b=2))        # mixed: 3
```

3 Positional: the order of the arguments decides which parameter each one fills.

4 Keyword: the name decides. The order of keyword arguments does not matter.

5 Mixed: positional arguments first, then keyword arguments.

Built-ins use keyword arguments too: print(1, 2, sep="-") and sorted(x, reverse=True).

Positional arguments must come before keyword arguments

KEYWORD FIRST

Never runs

```
1  def subtract(a, b):
2      return a - b
3  print(subtract(a=5, 2))
```

SyntaxError: positional argument follows keyword argument.

POSITIONAL FIRST

Works

```
1  def subtract(a, b):
2      return a - b
3  print(subtract(5, b=2))
```

3. The positional 5 fills a; the keyword fills b.

THE TRAP

This is a SyntaxError, so the whole file fails before line 1 runs, not just this call.

Argument errors are TypeErrors

CALL, WITH DEF SUBTRACT(A, B)	TYPEERROR MESSAGE
`subtract(5)`	missing 1 required positional argument: 'b'
`subtract(1, 2, 3)`	takes 2 positional arguments but 3 were given
`subtract(5, a=2)`	got multiple values for argument 'a'
`subtract(5, c=1)`	got an unexpected keyword argument 'c'

THE TIEBREAKER

Wrong count or wrong name: TypeError when the call runs. Wrong order: SyntaxError before anything runs.

A default fills in when the argument is left out

```
1  def power(base, exp=2):
2      return base ** exp
3  print(power(3))                 # 9:  exp uses its default
4  print(power(3, 3))              # 27: the default is replaced
5  print(power(exp=0, base=7))     # 1:  keywords, any order
```

1 exp=2 in the header gives exp a default value. base has none, so it is required.

3-4 Leave the argument out and the default is used; supply one and it wins.

Parameters with defaults must come last

DEFAULT FIRST

Never runs

```
1  def f(a=1, b):
2      return a + b
```

SyntaxError: parameter without a default follows parameter with a default.

DEFAULT LAST

Works

```
1  def f(b, a=1):
2      return a + b
3  print(f(5))
```

6. b is required; a is optional.

ON THE EXAM

The same shape as the call rule: required things first, optional things after.

Defaults are evaluated once, when def runs

```
1  n = 5
2  def show(x=n):          # the default is taken now: 5
3      return x
4  n = 10
5  print(show())           # 5, not 10
```

2 The default expression runs once, when the def statement runs, and the value is stored.

4-5 Changing n later does not change the stored default.

A list as a default is shared between calls

```
1  def add_item(item, box=[]):
2      box.append(item)
3      return box
4  print(add_item(1))      # [1]
5  print(add_item(2))      # [1, 2]: the same list again
```

1 One empty list is created, once, when def runs. Every call without box gets that same list.

5 The second call appends to the list the first call already changed.

The usual fix: box=None, then inside: if box is None: box = [].

Changing the object versus rebinding the parameter

CHANGE THE LIST

The caller sees it

```
1  def fill(items):
2      items.append(9)
3
4  data = [1]
5  fill(data)
6  print(data)
```

[1, 9]. items and data are two names for one list.

REBIND THE PARAMETER

The caller does not

```
1  def replace(items):
2      items = [9]
3
4  data = [1]
5  replace(data)
6  print(data)
```

[1]. items is moved to a new list; data is untouched.

THE TIEBREAKER

Methods, index assignment, del and += change the caller's list. Plain assignment to the parameter does not.

An immutable argument can never be changed from inside

```
1  def inc(n):
2      n += 1                  # makes a new int; rebinds n
3      return n
4  x = 5
5  print(inc(x), x)            # 6 5
6  def shout(s):
7      return s.upper()        # a new string
8  word = "hi"
9  print(shout(word), word)    # HI hi
```

2 An int cannot change, so += builds a new int and moves n. x keeps 5.

5 To get the new value out, the function must return it, and the caller must use it.

+= and del on a list parameter

```
1  def grow(items):
2      items += [0]              # extends the caller's list
3      del items[0]              # removes from the caller's list
4  data = [1, 2]
5  grow(data)
6  print(data)                  # [2, 0]
```

2 For a list, += changes the list in place, so the caller sees it. Module 12's rule.

3 del on an element also changes the shared list.

On a string argument, del text[2] is a TypeError: strings do not support item deletion.

What to carry into the exam

- Positional arguments first, then keyword. Otherwise: **SyntaxError**.
- Wrong count, a repeated parameter, or an unknown keyword: **TypeError** at run time.
- Defaults go last in the def, and are evaluated **once**, when def runs.
- A default list is **shared** between calls.
- A function can change a list it is given, but never the caller's variable, number or string.

What is the output of this program?

```
1  def calc(x, y, z):
2      return x * 10 + y - z
3
4  print(calc(1, z=2, y=5))
```

A 13

B 10

C 11

D The code raises a SyntaxError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def calc(x, y, z):
2      return x * 10 + y - z
3
4  print(calc(1, z=2, y=5))
```

x = 1 by position; z = 2 and y = 5 by name. 10 + 5 - 2 = 13.

A 13

Keywords fill by name, whatever their order.

B 10

Fills y and z in the order written: y = 2, z = 5.

C 11

Drops the y, or adds z instead of subtracting it.

D The code raises a SyntaxError

The positional argument comes first, so the order is legal.

THE SIGNAL

Keyword arguments are matched by name. The order in which they are written does not matter.

What happens when this program runs?

```
1  def area(w, h):
2      return w * h
3
4  print("start")
5  print(area(w=3, 4))
```

A start, then 12

B start, then 7

C start, then a TypeError

D Nothing is printed; the program raises a SyntaxError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  def area(w, h):
2      return w * h
3
4  print("start")
5  print(area(w=3, 4))
```

SyntaxError: positional argument follows keyword argument. Found before line 1 runs.

A start, then 12

The call is never reached. The file does not compile.

B start, then 7

** is multiplication, and the call never runs anyway.*

C start, then a TypeError

Wrong order is a SyntaxError, found before anything runs.

D Nothing is printed; the program raises a SyntaxError

A positional after a keyword is a grammar error, so start never prints.

THE SIGNAL

Argument order errors are SyntaxErrors. Argument count errors are TypeErrors.

What is the output of this program?

```
1  def price(cost, tax=20, discount=0):
2      return cost + cost * tax // 100 - discount
3
4  print(price(100, discount=5), price(100, 10))
```

A 105 110

B 115 120

C 115 110

D 95 110

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def price(cost, tax=20, discount=0):
2      return cost + cost * tax // 100 - discount
3
4  print(price(100, discount=5), price(100, 10))
```

First: tax 20 (default), discount 5: 100 + 20 - 5 = 115. Second: tax 10, discount 0: 110.

A `105 110`

Puts the 5 into tax. discount=5 names its parameter.

B `115 120`

Keeps tax at 20 in the second call. The positional 10 replaces it.

C **`115 110`**

Defaults fill the gaps; the positional 10 goes to tax, the second parameter.

D `95 110`

Forgets the default tax of 20 in the first call.

THE SIGNAL

A keyword argument skips over earlier defaults. A second positional argument fills the second parameter.

What is the output of this program?

```
1  def change(a, b):
2      a.append(3)
3      b = [0]
4
5  x = [1]
6  y = [2]
7  change(x, y)
8  print(x, y)
```

A `[1, 3] [0]`

B `[1, 3] [2]`

C `[1] [2]`

D `[1] [0]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def change(a, b):
2      a.append(3)
3      b = [0]
4
5  x = [1]
6  y = [2]
7  change(x, y)
8  print(x, y)
```

a shares x's list and changes it. b is rebound; y is untouched.

A `[1, 3] [0]`
Treats b = [0] as changing y. It only moves the name b.

B `[1, 3] [2]`
append changes the shared list; assignment rebinds only b.

C `[1] [2]`
Assumes a function works on copies. It gets the same objects.

D `[1] [0]`
Reverses both effects.

THE SIGNAL

Inside a function: change the object and the caller sees it; rebind the parameter and the caller does not.

What is the output of this program?

```
1  def bump(n):
2      n += 5
3      return n
4
5  n = 1
6  bump(n)
7  print(n, bump(n))
```

A 6 6

B 1 1

C 6 11

D 1 6

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def bump(n):
2      n += 5
3      return n
4
5  n = 1
6  bump(n)
7  print(n, bump(n))
```

The parameter n is not the outer n. Line 6's result is thrown away.

A 6 6

Assumes line 6 changed the outer n. Ints cannot be changed from inside.

B 1 1

bump does return 6; the second value is that return.

C 6 11

Assumes both calls changed the outer n.

D 1 6

The outer n stays 1; bump(1) returns 6 again.

THE SIGNAL

Same name inside and outside does not mean the same variable. An int argument cannot be changed.

What is the output of this program?

```
1  limit = 3
2  def cap(x, top=limit):
3      return min(x, top)
4
5  limit = 100
6  print(cap(50))
```

A 3

B 50

C 100

D The code raises a NameError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  limit = 3
2  def cap(x, top=limit):
3      return min(x, top)
4
5  limit = 100
6  print(cap(50))
```

top's default was stored as 3 when def ran. min(50, 3) is 3.

A **3**
The default was taken when def ran, while limit was 3.

B **50**
Would need top to be 100 or more.

C **100**
Assumes the default is looked up on each call.

D **The code raises a NameError**
limit exists when def runs, so the default is fine.

THE SIGNAL

A default is evaluated once, when def runs. Later changes to the names it used have no effect.

What is the output of this program?

```
1  def log(msg, history=[]):
2      history.append(msg)
3      return len(history)
4
5  log("a")
6  print(log("b"), log("c", []))
```

A 1 1

B 2 1

C 2 3

D 3 1

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def log(msg, history=[]):
2      history.append(msg)
3      return len(history)
4
5  log("a")
6  print(log("b"), log("c", []))
```

The default list holds "a", then "a", "b". The call with [] gets its own fresh list.

A 1 1

Assumes a fresh default list on every call. It is created once.

B 2 1

a and b share the default list; c gets the list passed in.

C 2 3

Adds c to the shared list. An explicit argument replaces the default.

D 3 1

Counts c in the shared list too.

THE SIGNAL

Calls that rely on a default list share it. A call that passes its own list does not touch it.

> def f(a, b=1): pass has been defined. Each call is run on its own.

Which two calls raise a TypeError? (Choose two.)

A `f(1)`

B `f(b=2, a=1)`

C `f(1, 2, 3)`

D `f(1, a=2)`

◆ PAUSE AND DECIDE

Which two calls raise a TypeError? (Choose two.)

A `f(1)`

a is 1 and b uses its default. Fine.

B `f(b=2, a=1)`

Both by name, any order. Fine.

C `f(1, 2, 3)`

Takes from 1 to 2 positional arguments but 3 were given.

D `f(1, a=2)`

1 fills a by position, then a=2 fills it again: multiple values.

THE SIGNAL

Count the arguments against the parameters, and check no parameter is filled twice.